SQL OPTIMIZATION

PROBLEM 3

WHEN FUNCTIONS BREAK INDEXES

Applying functions to columns kills performance.

INDEX_KILLERS.SQL

```sql
-- 🔴 Searching for email domains
SELECT * FROM users
WHERE SUBSTRING(email, ...) = 'gmail.com';

-- 🔴 Searching for last month's orders
SELECT * FROM orders
WHERE YEAR(order_date) = 2024
AND MONTH(order_date) = 12;

-- 🔴 Wildcard at the start
SELECT * FROM products
WHERE product_name LIKE '%pro%';
```

Anandnarayanan S

Follow for more SQL skills

PATTERN

IS IT SARGABLE?

Search ARGument ABle queries allow Index Seeks.

NON-SARGABLE

FUNC(Column) = Value

The DB runs the function on **EVERY SINGLE ROW** to check the result.

Result: **Index Scan (Slow)**

SARGABLE

Column = Value

The column is clean. The DB traverses the B-Tree directly.

Result: **Index Seek (Fast)**

Anandnarayanan S

Follow for more SQL skills

SOLUTION

THE INDEX-FRIENDLY FIX

Computed Columns & Range Queries.

OPTIMIZED_SARGABLE.SQL

```sql
-- ✅ 1. Computed Column for Emails
ALTER TABLE users
ADD email_domain AS SUBSTRING(...) PERSISTED;
CREATE INDEX idx_domain ON users(email_domain);

SELECT * FROM users
WHERE email_domain = 'gmail.com';

-- ✅ 2. Range Query for Dates
SELECT * FROM orders
WHERE order_date ≥ '2024-12-01'
AND order_date < '2025-01-01';
```

Anandnarayanan S

Follow for more SQL skills

SUMMARY

SARGABLE CHECKLIST

✓ **Keep Columns Clean**
Avoid functions on the Left-Hand Side of WHERE.

✓ **Use Computed Columns**
If you must parse strings, store the result.

✓ **Date Ranges**
Use >= and < operators instead of YEAR() or MONTH().

FOUND THIS HELPFUL?

REPOST THIS!

Anandnarayanan S
Follow for more SQL skills